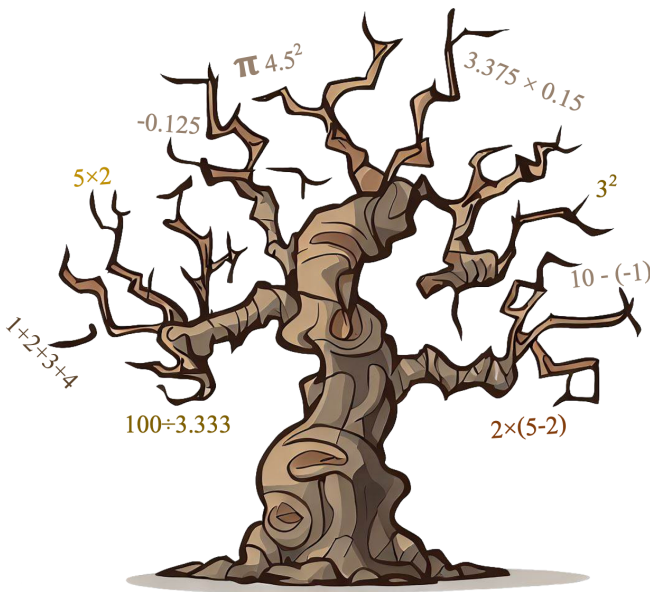




Introduction

Simple arithmetic expressions of the form

$$value_1 \text{ op } value_2$$

are easy to represent as a C data structure:

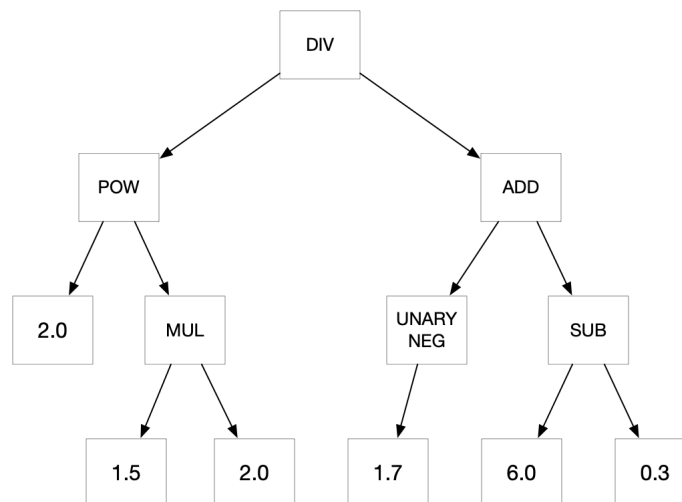
```
struct simple_expression {
    double value1;
    char op;
    double value2;
};
```

The more general case, where arithmetic expressions can have any level of nested parentheses and operators appearing in any order, is less straightforward to represent in C. For instance, consider the following expression:

$$2^{(1.5 \times 2)} \div (-1.7 + (6.0 - 0.3))$$

In this assignment, we will create a module I am calling Expression Tree, or ExprTree. We will create a straightforward *dynamically allocated tree data structure* that can represent arithmetic expressions to any arbitrary level of nesting. Values in our tree will be stored as doubles, and will be represented as leaf nodes in our tree. Meanwhile, the five binary operators $+$, $-$, $\times$, $\div$, and $^$ will be stored as an enum, and will be represented as interior nodes. There is also a unary negation operator to handle the math operation $-a$, which will also be represented as an interior node, but with only one child.

The example expression above is represented in ExprTree as follows:



The following code shows how a caller might use the ExprTree module:

```
ExprTree tree = NULL;
char buffer[128];
double result;

// 6.5 * (4 + 3)
tree = ET_node(OP_ADD, ET_value(4.0), ET_value(3.0));
tree = ET_node(OP_MUL, ET_value(6.5), tree);

ET_tree2string(tree, buffer, sizeof(buffer));
result = ET_evaluate(tree);
printf("%s ==> %g\n", buffer, result);

ET_free(tree);

// -0.125 (as a value)
const double value = -0.125;
tree = ET_value(value);

ET_tree2string(tree, buffer, sizeof(buffer));
result = ET_evaluate(tree);
printf("%s ==> %g\n", buffer, result);

ET_free(tree);

// -(-0.125) (using value and unary negation)
tree = ET_node(UNARY_NEGATE, ET_value(value), NULL);

ET_tree2string(tree, buffer, sizeof(buffer));
result = ET_evaluate(tree);
printf("%s ==> %g\n", buffer, result);

// double unary negation: (- (- (-0.125)))
tree = ET_node(UNARY_NEGATE, tree, NULL);

ET_tree2string(tree, buffer, sizeof(buffer));
result = ET_evaluate(tree);
printf("%s ==> %g\n", buffer, result);

ET_free(tree);

// 2^3 / (1.3 + 2.7)
tree = ET_node(OP_ADD, ET_value(1.3), ET_value(2.7));
tree = ET_node(OP_DIV, ET_node(OP_POWER, ET_value(2), ET_value(3)), tree);

ET_tree2string(tree, buffer, sizeof(buffer));
result = ET_evaluate(tree);
printf("%s ==> %g\n", buffer, result);

ET_free(tree);

// (20 - 10) - (30 - 25)
tree = ET_node(OP_SUB, ET_value(20), ET_value(10));
tree = ET_node(OP_SUB, tree, ET_node(OP_SUB, ET_value(30), ET_value(25)));
```

```

ET_tree2string(tree, buffer, sizeof(buffer));
result = ET_evaluate(tree);
printf("%s ==> %g\n", buffer, result);

ET_free(tree);

```

That code produces the following output:

```

(6.5 * (4 + 3)) ==> 45.5
-0.125 ==> -0.125
(--0.125) ==> 0.125
- (--0.125) ==> -0.125
((2 ^ 3) / (1.3 + 2.7)) ==> 2
((20 - 10) - (30 - 25)) ==> 5

```

Our Data Structure

Our ExprTree implementation will be based the following data structure:

```

typedef enum {
    VALUE,
    UNARY_NEGATE,
    OP_ADD,
    OP_SUB,
    OP_MUL,
    OP_DIV,
    OP_POWER
} ExprNodeType;

#define LEFT 0
#define RIGHT 1

struct _expr_tree_node {
    ExprNodeType type;
    union {
        struct _expr_tree_node *child[2];
        double value;
    } n;
};

```

The `struct _expr_tree_node` has a different data field depending on the node type. A leaf node has the `type` field set to `VALUE`, and stores its value in `n.value`. Meanwhile an interior node has the `type` field set to anything other than `VALUE`, and stores its children in `n.child[LEFT]` and `n.child[RIGHT]`. (For `UNARY_NEGATE` nodes, only `n.child[LEFT]` is used.)

Each node is created by a call to either `ET_value` (for leaf nodes) or `ET_node` (for interior nodes), and destroyed by a call to `ET_free` (which recursively descends the entire ExprTree and frees all nodes in one call).

Assignment Detail

The ExprTree library has only seven functions, which you will write:

Function	Description
ET_value	Create and return a new leaf node
ET_node	Create and return a new interior node
ET_free	Destroy an entire ExprTree, calling free() on all previously malloc'd memory
ET_count	Return the number of nodes in an ExprTree
ET_depth	Return the maximum depth of an ExprTree
ET_evaluate	Evaluate an ExprTree and return the resulting value as a double
ET_tree2string	Convert an ExprTree into a printable ASCII string

Each of these functions is fully documented in the header file `expr_tree.h`. The last five functions in the list above should be implemented recursively.

I have provided the following files for you:

- **expr_tree.h** – the header file which documents the interface to all the functions. You should not need to modify this file.
- **expr_tree.c** – the implementation file. You should add your code as indicated in the comments. You should not need to modify the code I have provided, but you are free to do so.
- **et_test.c** – This file contains automated tests for the ExprTree module. As distributed, it has some tests, but you should add additional tests, particularly for the `ET_tree2string` function.
- **Makefile** – A makefile to build the project. You should be able to type 'make' from the command line in your working directory, and Make will compile the executable `et_test`.

Implementation Notes

For this assignment, we will again assume that malloc always returns a valid memory block. You should ensure this by calling assert immediately after every call to malloc, but you do not otherwise need to worry about handling the out-of-memory error case.

As with all things in software engineering, the best approach is to break the problem down into the smallest steps you can think of, and then implement one step at a time, testing as you go.

To that end, the functions are listed above in rough order of difficulty to code, so you may wish to approach them in that order.

The function `ET_tree2string` is particularly challenging, once you start worrying about the `buf_sz` constraint. I recommend you first get this function working fully *without worrying about `buf_sz`*. Then go back and add the bounds checking needed to address `buf_sz`.

Also note the following rules for `ET_tree2string`, which are needed to ensure that your implementation matches what our automated tests expect:

- Use the `%g` format specifier to `snprintf` to convert doubles to strings.
- `ET_tree2string` should emit exactly one set of parentheses per interior node of the `ExprTree`, and no parentheses for leaf nodes.

I have again added `-fsanitize=address` to the gcc flags in the Makefile. As before, your code must be free of memory leaks.

Testing

A large part of this assignment is for you to think through not just how to implement your code, but how to *prove to yourself and everyone else* that your implementation is solid. Therefore, I want YOU to write good test cases. When writing your test cases, try to think through all the possible corner cases. You may wish to review the class material and perhaps the tests I provided for Assignment 4.

We have again made `scottycheck` available, but I do not want you to use it in place of your own tests. Therefore `scottycheck` use is throttled to once every 20 minutes. To run:

```
scottycheck isse-08 <list all your .c and .h files>
```

You should not attempt `scottycheck` until you are passing your own test cases. If `scottycheck` indicates failures that are not caught by your own tests, you should spend time thinking about how to enhance your tests (as well as fixing whatever problems `scottycheck` identified).

Grading and Submission

To submit your code, you should create a private GitHub repo that contains your code and a `README.md` file; the `README` should (at minimum) document how to build your code. Prior to the submission deadline, you should then submit the URL of the GitHub repo on Canvas. We will compute a grade based on the state of your repo as of the submission deadline, so feel free to make changes to it up to that point.

Points will be awarded as follows:

Points	Item
5	The basics: Did you submit your GitHub URL in your Canvas submission? Does your repo contain only true

Points	Item
	sources and not derived files? Does your code compile without warnings? [Points are all-or-nothing.]
11	Code legibility: Is your code indented reasonably and are variable names well selected? Are there appropriate comments? Does the code use #defines appropriately? Subjectively, does this look like good, professional code?
64	Correctness: Based on running the automated tests used by scottycheck [8 tests, 8 points per test]
10	Memory leaks: Does your code have memory leaks as reported by -fsanitize=address? [score is computed as $10 - \text{num_blocks_leaked} \times 2$]
10	Testing: Did you add enough test cases to provide strong test coverage?

Good luck and happy coding!